
SILENT FAILURES BEYOND THE 32-BIT INDEX RANGE: A DIFFERENTIAL CHARACTERIZATION OF LARGE-TENSOR MATRIX MULTIPLICATION IN PYTORCH’S MPS BACKEND

A PREPRINT

 **Junichiro Niimi***
Meijo University

ABSTRACT

Apple Silicon machines with large unified memory make it possible to hold large tensors on a desktop GPU. However, we found that PyTorch’s Metal Performance Shaders (MPS) backend silently returns wrong results for batched matrix multiplication with more than 2^{32} elements. `torch.bmm`, including its wrappers `torch.matmul` and eager attention, returns relative errors above 1 without an exception or a warning in every PyTorch release tested (2.4.1 to 2.14.0). We sweep `bmm` over dtypes, memory layouts, shapes and batch sizes around 2^{31} and 2^{32} elements, and judge every result against a float64 computation on the CPU. Three rules account for every outcome on 2.14.0. When the output exceeds 2^{32} elements and an operand is a transposed view, the entire output is wrong and equals a computation that ignores that operand’s strides. Otherwise, a view with at least 2^{31} elements raises an exception, and a contiguous input above 2^{32} elements makes exactly the batches beyond that point wrong, equal to a computation whose index wraps at 2^{32} . A slightly larger problem can thus turn an explicit error into a silent failure. The rules extend to the backward pass, where a correct forward pass can return silently wrong gradients. A second machine with another chip, under two macOS versions, reproduces all 6156 results, including the wrong values, and the same sweeps on an NVIDIA A100 are correct in all 2530 runs. In a public sentiment classifier, one oversized batch corrupts a third of the outputs, which collapse onto one class. All findings come from observable behavior, without access to the backend’s closed-source kernels; we release the harness, raw results and a guard that stops any MPS operation touching 2^{32} or more elements at <https://github.com/jniimi/mps-silent-failures>.

1 Introduction

1.1 Background

With unified memory, the CPU and the GPU of a machine share a single pool of memory, so a model and its data can be placed and moved without the fixed capacity of a separate GPU memory. This flexibility matters more as models, contexts and batches grow, and it has made a desktop machine a practical place to build and analyze large models. Apple Silicon machines offer 192 GB or more of such memory, and a single fp32 tensor with 2^{32} elements (17 GB) fits with room to spare. Ordinary workloads reach this size: attention over long contexts or large batches, whose score matrices grow with the square of the sequence length (Tay et al., 2022), feature extraction for probing (Hewitt and Manning, 2019), and attention-map analysis (Clark et al., 2019; Abnar and Zuidema, 2020). For example, attention scores of shape $(B \cdot H, L, L)$ pass 2^{32} elements at 32 heads, a context of 4096 tokens and a batch of 9.

PyTorch (Paszke et al., 2019) runs on Apple GPUs through its MPS backend. On macOS 14, batched matrix multiplication whose output exceeded 2^{32} elements stopped with an explicit error (Tiling of batch matmul for larger than 2^{32} entries only available from MacOS15 onwards). After an upgrade of the same machine to macOS 27.0, the same operation runs to completion but returns wrong values, without any error or warning (Niimi,

*jniimi@meijo-u.ac.jp

2026b), and we find further silent failures nearby, among them gradients that are silently wrong although the forward pass is correct. For this operation, upgrading the operating system turned a hard failure into a silent one.

Silent failures are among the most costly framework bugs since nothing in the pipeline flags them (Tampon et al., 2023). In the case study of Section 4.5, overall accuracy drops by only 7 points while recall for two of the three classes falls to almost zero on the affected inputs, and no error, warning or non-finite value appears.

1.2 Contributions

Our contributions are as follows:

- **A method for characterizing failures at a size boundary from the outside.** A boundary-targeted differential sweep of `bmm` on the MPS backend varies the dtype, the memory layout, the shape, the PyTorch release (11 versions) and which tensor crosses a candidate boundary (2^{31} elements, 2^{32} elements, 2^{32} bytes), and judges every result against a float64 computation on the CPU. Index-encoded inputs let us read off, without access to the implementation, which input batch, or which row and column within a batch, a wrong output element was computed from.
- **A characterization of the failures.** Three rules, applied in order, reproduce all 1376 outcomes with random inputs, including series that probe conditions outside the sweep from which the rules were derived: inputs and output above 2^{32} elements together, batch sizes near $2 \cdot 2^{32}$ elements, and small views at storage offsets beyond 2^{32} elements. The wrong values are identified as two specific misreadings of the inputs. The same rules, applied to the two products that define the gradients, reproduce all 592 outcomes of a sweep of the backward pass, in which a correct forward pass can come with silently wrong gradients.
- **Evidence that the failures are deterministic and specific to the backend.** Every series is repeated on a second machine with a different chip, under macOS 26.6.2 and again under macOS 27.0; the three configurations agree in all 6156 common runs, down to the value of the largest error, which separates the chip from the macOS version. The same sweeps on CUDA, including the backward pass, are correct throughout. A first-pass suite of 26 test cases on the 11 releases places `bmm` among other operations on large tensors and records a regression in 2.14.0, where `torch.arange` stops raising and silently truncates.
- **Impact and mitigation.** In a public sentiment classifier, a single oversized batch corrupts a third of the outputs without any error, the affected predictions collapse onto one class, and chunked execution of the same inputs is correct. A drop-in guard stops any MPS operation that would create or read a tensor with 2^{32} or more elements; we report its overhead and evaluate it against every recorded outcome.

2 Related Work

Bugs in deep learning frameworks. Empirical studies classify the bugs reported against deep learning frameworks by symptom and root cause (Chen et al., 2023). Tampon et al. (2023) study *silent* bugs in Keras and TensorFlow, which produce wrong behavior without a crash or an error message, and find that they are hard for users to notice and to trace back to the framework. Wang et al. (2022) study numerical bugs in deep learning programs, such as overflow and loss of precision in floating-point computation. The failures we report are silent in the sense of Tampon et al., but they are not rounding problems: the relative error is of order one and appears only above a fixed element count. These studies classify faults after the fact, from issue trackers and repositories. We instead take a single reported issue (Niimi, 2026b) as the starting point for controlled measurements that establish where the failure occurs, under which conditions, and what the wrong values are.

Testing deep learning libraries. Without a ground-truth oracle, library testing is usually differential. CRA-DLE (Pham et al., 2019) runs the same model on several Keras backends and flags inconsistent outputs. Fuzzers generate test inputs automatically: FreeFuzz (Wei et al., 2022) mines API calls from documentation, tests and models, and compares CPU with GPU execution; NNSmith (Liu et al., 2023b) builds valid computation graphs; generation-based differential fuzzing (Liu et al., 2023a) generates whole models and compares libraries on them; TitanFuzz (Deng et al., 2023) lets a large language model write the test programs; and ∇ Fuzz (Yang et al., 2023) tests automatic differentiation by comparing outputs and gradients across execution scenarios. The sweep in this paper (Section 3) is differential as well, comparing MPS with a float64 result on the CPU, with CUDA only as a control, and its case study (Section 4.5) uses a metamorphic relation (Chen et al., 2018) between one large batch and its chunks. The difference lies in what is varied. The tools above aim at breadth over APIs and operator combinations and need each test to be cheap, while a single fp32 tensor with 2^{32} elements already takes 17 GB. To our knowledge, none of them targets element-count boundaries, and none uses the MPS backend as the system under test. We fix a small family of operations and vary size, memory layout and library version instead.

Integer width and size boundaries. Integer overflow is common even in mature, widely used programs, and developers often misunderstand it (Dietz et al., 2015). Wressnegger et al. (2016) show that the migration to 64-bit platforms created vulnerabilities that had not existed before: wider integer types and a larger addressable memory let values reach sizes that code written with 32-bit quantities in mind does not handle. Our setting has the same shape, since a machine with 192 GB of unified memory is what makes a tensor with 2^{32} elements an ordinary object. Closer to numerical software, the count argument of MPI routines is a C `int`, so that messages with 2^{31} or more elements are hard to express, and a library that adds support for large counts exposed bugs in MPI implementations (Hammond et al., 2014). The candidate boundaries of Section 3 are of this kind: they are chosen from the widths of integer types, and the test points are placed around them.

Reproducibility across hardware and silent data corruption. Training results vary between runs and between accelerators because of nondeterministic kernels and differences in floating-point reduction order (Pham et al., 2020; Zhuang et al., 2021). Such variation is small per operation and is expected behavior, yet it can grow over training: Pham et al. (2020) report that implementation-level nondeterminism alone changes overall accuracy across identical training runs by up to 2.9%, and per-class accuracy by up to 52.4%. Zhuang et al. (2021) likewise find a far larger effect on per-class metrics and on subgroups of the data than on top-line accuracy. At the other extreme, studies of large fleets report silent data corruption caused by defective processors, which affects individual machines and is often intermittent (Dixit et al., 2021; Hochschild et al., 2021). The failures in this paper belong to neither group. They are deterministic, they reproduce from a few lines of code across 11 PyTorch releases, and in our measurements whether they occur is decided by tensor size, memory layout and library version, so they are software faults rather than noise or hardware defects.

3 Method

A black-box study. PyTorch’s MPS backend is open source, but it is built on Apple’s Metal Performance Shaders framework, whose kernels are closed source. We therefore treat the backend as a black box: every statement in this paper is derived from inputs that we control and outputs that we observe, judged against a reference that does not involve the GPU. This is the only view of the system that its users have, and everything we measure can be repeated by anyone with an Apple Silicon machine and the public PyTorch wheels. What the method cannot deliver is a root cause; when we say that a wrong output *equals* a particular misreading of the inputs, we state an agreement of values, not a finding about the implementation.

From `matmul` to `bmm`. `torch.matmul` (the `@` operator) dispatches on the number of dimensions. For inputs with three or more dimensions it broadcasts the leading batch dimensions, folds them into one, and calls `torch.bmm` on (B, n, m) and (B, m, p) tensors. The attention mechanism originates in neural machine translation (Bahdanau et al., 2015). The form used in current models is the scaled dot-product attention of the Transformer (Vaswani et al., 2017), $\text{softmax}(QK^\top/\sqrt{d})V$. The Hugging Face Transformers library (Wolf et al., 2020) calls the implementation that evaluates this formula step by step *eager* attention (`attn_implementation="eager"`)¹, in contrast to fused kernels such as FlashAttention (Dao et al., 2022), which never return the score matrix. Eager attention is therefore the textbook computation and not a special variant. It computes $QK^\top$ and then $\text{softmax}(QK^\top)V$ with `matmul`, so both products go through `bmm`, and the scores of shape (B, H, L, L) exist as a tensor. `nn.Linear` instead flattens its input to two dimensions and calls `addmm`, and fused `scaled_dot_product_attention` (SDPA) is a single operator that does not return the score matrix to the caller.

Environment. The MPS results come from two machines (Table 1). Machine A runs every series. Machine B, with a different chip and memory size, repeats every series twice with the same scripts, identified by their SHA-256 hashes in the manifests: first under macOS 26.6.2, and again after the machine was upgraded to macOS 27.0, the version of machine A. Machine A against machine B under macOS 27.0 thus varies the hardware alone, and the two passes of machine B vary the macOS version alone. Results are compared between configurations run by run, in every recorded field except the run identifiers, the description of the environment, the measured times and memory use, and the text that the system writes when a process aborts. PyTorch versions are installed from the official PyPI wheels into fresh environments with `uv`, one environment per version.

Axes. The sweep targets one operation, `torch.bmm` on a of shape (B, M, K) and b of shape (B, K, N) , and varies the axes in Table 2. Every shape has 2^{16} elements per batch in the tensor that is meant to cross the boundary (the

¹https://huggingface.co/docs/transformers/attention_interface

Table 1: Test environments. Machines A and B produce the MPS results. Machine C is the hosted runtime (Google Colab) of the CUDA control; its values are those recorded in the manifests, which are identical in all sessions.

	machine A	machine B	machine C (CUDA control)
Model	Mac Studio (Mac14,14)	MacBook Pro (Mac14,6)	Google Compute Engine VM
Chip	Apple M2 Ultra (24-core CPU, 76-core GPU)	Apple M2 Max (12-core CPU, 38-core GPU)	Intel Xeon @ 2.20 GHz (12 vCPUs), NVIDIA A100-SXM4-80GB
Memory	192 GB unified	96 GB unified	167 GB host, 80 GB GPU
OS	macOS 27.0 (build 26A428)	macOS 26.6.2 (build 25G83), then macOS 27.0 (build 26A428)	Ubuntu 24.04.1 LTS, driver 580.82.07, CUDA 13.0
Python	3.12.12	3.12.11	3.12.3
PyTorch	the 11 versions (2.4.1, 2.5.1, 2.6.0, 2.7.1, 2.8.0, 2.9.1, 2.10.0, 2.11.0, 2.12.1, 2.13.0, 2.14.0)	the same 11 versions, under each macOS version	2.14.0+cu130

Table 2: Axes of the bmm sweeps. In the main sweep, the base shapes are run in both dtypes and with all input kinds, and the additional shapes in fp32 with random inputs. Entries marked *ext.* belong to the extension series and *bwd.* to the backward series, which are described below.

axis	values
shape (M, K, N)	output-side: base=(256, 64, 256), (512, 64, 128), (128, 64, 512) input-side: base=(256, 256, 64), (512, 128, 64), (128, 512, 64) inputs and output: (256, 256, 256) (<i>ext.</i>)
operation	forward; forward and backward (<i>bwd.</i> , base shapes)
dtype	fp32, fp16
layout	contiguous; <i>b</i> transposed; <i>a</i> transposed (<i>a</i> contiguous storage of the transposed values, passed as a <code>transpose(1, 2)</code> view); sliced (both operands are <code>[1:]</code> of a storage with one extra leading batch, so the storage offset is one batch); offset (<i>ext.</i> : a view of 4096 batches behind <i>O</i> leading batches, for both operands)
input values	random (uniform on $[-1, 1]$, zeros replaced so that every element is non-zero); index-encoded (see below)
batch size B	4096 to 65538 (see below); 131071 to 131073 (<i>ext.</i> : far points, base shapes)
PyTorch	2.14.0 for the main sweep, the extension series and the backward series; 2.4.1 to 2.13.0 for a reduced sweep (base shapes only)

crossing tensor), which is the output ($MN = 2^{16}$) in the output-side shapes and the input a ($MK = 2^{16}$) in the input-side shapes. That tensor therefore has $2^{16}B$ elements. In the base shapes the other two tensors have $2^{14}B$ elements each and stay near 2^{30} . The additional shapes split the same number of elements into different (M, K, N) , to check that the behavior depends on the element count and not on a particular dimension. In each of them one of the other tensors has $2^{15}B$ elements and reaches 2^{31} at $B = 65536$. The layouts change only how the operands are stored. The logical values of a and b are generated on the CPU from a fixed seed per batch, independently of the layout, and the storage is built from them, so that for the same seed all four layouts compute the same matrix product and their results can be compared directly. Operations other than bmm are covered only by the first-pass suite (Section 4.6).

Boundaries and search. We consider three candidate boundaries for the crossing tensor: 2^{31} elements ($B_0 = 32768$), 2^{32} elements ($B_0 = 65536$), and 2^{32} bytes ($B_0 = 16384$ in fp32; in fp16 this coincides with 2^{31} elements). These candidates are hypotheses about where a limit could lie. A 32-bit signed integer can index at most $2^{31} - 1$ elements and an unsigned one at most $2^{32} - 1$, so an implementation that holds an element index in either type would change its behavior at the first or the second candidate. The third candidate tests whether a limit applies to the number of bytes instead of the number of elements: in fp32 a tensor reaches 2^{32} bytes at 2^{30} elements, and a byte limit would also make fp16 and fp32 switch at different batch sizes. Each series, that is, each combination of shape, dtype and layout, is run at the five consecutive batch sizes $B_0 - 2, \dots, B_0 + 2$ around every candidate, at a small baseline point ($B = 4096$, 2^{28} elements), and on an evenly spaced grid between these points (seven values per interval for the base shapes and three for the additional ones). This gives 37 batch sizes per series in fp32 and 25 in fp16 for the base shapes. A change of behavior that does not coincide with a candidate would appear as two neighboring grid points

with different outcomes, and the driver then bisects between them down to a single batch. At each switch point, the batch sizes on both sides are run again with a comparison over all elements and three seeds; a series without a switch receives this comparison at its largest batch size.

Reference and comparison. The reference is the same product computed in float64 on the CPU. To build it, the logical inputs of each batch are generated again on the CPU from their seeds; inputs are never read back from the device for this purpose, so a fault in the transfer cannot cancel out. The device output is copied to the CPU in chunks, compared, and discarded. Let Y_i be batch i of the device output and Y_i^* the same batch of the reference, and let $\|A\|_{\max}$ denote the largest absolute entry of a matrix A . The error of batch i is

$$e_i = \frac{\|Y_i - Y_i^*\|_{\max}}{\|Y_i^*\|_{\max}}, \quad (1)$$

and batch i is wrong when $e_i > \tau_d$, where the tolerance τ_d is set for each dtype d and each series. Let $\hat{e}_d$ be the largest e_i in a set of calibration runs far below any candidate (every shape of the series that is run in that dtype, $B = 4096$, contiguous, three seeds, all elements). Then

$$\tau_d = \max(10 \hat{e}_d, \tau_d^{\min}), \quad (2)$$

with a floor $\tau_d^{\min}$ of 10^{-6} in fp32 and 10^{-3} in fp16.

A comparison is either *full*, over all batches, or *sampled*. A sampled comparison takes 256 evenly spaced batches, the first 4 and the last 8, and the batches around every position where the flat index of an operand or of the output passes a multiple of a candidate boundary, which gives 266 to 281 batches. The grid and the points around the candidates use sampled comparisons, and the switch points use full comparisons with three seeds. The reduced sweep for earlier PyTorch versions uses a coarser grid (two values per interval) and one seed for the full comparisons.

Each run is classified as *ok*; *error*, when an exception is raised; *crash*, when the process ends without returning a result; *timeout* (20 minutes for a sampled and 60 for a full comparison); *wrong*; *truncated* (an output of zeros), when at least 90% of the wrong elements are exactly zero, which the non-zero inputs make detectable; or *input corrupt*, when the inputs read back from the device in the sampled batches differ from the generated values, which would point to the transfer and not to bmm. Runs whose estimated memory exceeds 60% of the physical memory are skipped. For wrong runs we also test a small set of hypotheses about how the inputs were read. For up to 16 wrong batches at each end of the wrong range, we recompute on the CPU the output that would result if the strides of a transposed operand were ignored, if the flat index into an operand wrapped around at 2^{32} elements, if both happened, or if the storage offset were ignored, and record the fraction of wrong elements that each hypothesis reproduces.

Extensions of the sweep. Three extension series use the same worker, the same reference and the same classification as the main sweep, and address conditions that it does not reach. (i) *Inputs and output together.* The shape $(M, K, N) = (256, 256, 256)$ gives a, b and the output 2^{16} elements per batch each, so that both inputs and the output pass every candidate at the same batch size. It is swept like a base shape, in fp32 and fp16 and in the four layouts. (ii) *Far points.* For the two base shapes, all layouts and both dtypes, we add the batch sizes $2B_0 - 1$, $2B_0$ and $2B_0 + 1$ with $B_0 = 65536$, where the crossing tensor has about $2 \cdot 2^{32}$ elements, with a comparison over all elements and with the four index-encoded inputs at $2B_0 + 1$. (iii) *Large storage offsets.* To separate the size of a view from its position in memory, a small view of $B = 4096$ batches (2^{28} elements in the crossing tensor) is placed behind O leading batches in a larger storage, for both operands. Leading batch j is filled with a value that is constant within the batch and differs between batches and from the values of the view, so that a read from the wrong place is detectable. O is swept so that the storage offset of the larger operand passes 2^{31} and 2^{32} elements: $O = 1$, five consecutive values around each candidate, and a grid in between (27 values in fp32, 19 in fp16), all with comparisons over all elements. The hypotheses of wrapped index and ignored offset are adapted to this layout. The tolerance is calibrated separately for each series and is reported with its results.

Backward pass. To test gradients, the worker runs the forward product $y = \text{bmm}(a, b)$ with both operands requiring gradients and then backpropagates an upstream gradient G of the shape of y . We write A_i, B_i, Y_i and G_i for batch i of a, b, y and G , and ℓ for the scalar that is differentiated. G is generated on the CPU from its own seeds in the same way as an operand and is contiguous on the device. The references are computed in float64 on the CPU from the regenerated inputs, batch by batch,

$$Y_i^* = A_i B_i, \quad \left(\frac{\partial \ell}{\partial a}\right)_i^* = G_i B_i^\top, \quad \left(\frac{\partial \ell}{\partial b}\right)_i^* = A_i^\top G_i, \quad (3)$$

and the output and the two gradients are each compared with the error of Eq. (1) and the same tolerance, which is calibrated on the largest error of the three tensors in the calibration runs of this series. A run is classified by the worst

of the three tensors, and the class, the largest error and the wrong batches are also recorded per tensor; for an exception we record whether it was raised in the forward or in the backward pass. The layout applies to a and b . This series uses the two base shapes, both dtypes, the four layouts and the batch sizes of the main sweep.

CUDA control. The main sweep, the backward series and the three extension series are repeated on an NVIDIA A100 (machine C in Table 1) with the same driver and worker scripts; the worker selects the device with a flag, so that the input generation, the reference, the error of Eq. (1) and the classification are identical; the SHA-256 hashes of both scripts are recorded in every manifest and equal those of the MPS runs. TF32 is disabled. Because a hosted session does not last for the whole sweep, each series is split, where necessary, into chunks of one shape, one dtype and one or two layouts (the far points by shape and dtype), each chunk is one run, and only chunks that finished are used. Every chunk is calibrated on its own shape and dtype with Eq. (2), so the tolerance differs between chunks, and for the control we also report the largest error over all runs. The batch sizes are the same 42 values. On CUDA no switch point exists by construction of the search, so the full comparisons fall on the largest batch size of every series. These runs serve as a control and not as an oracle.

Index-encoded inputs. Random inputs show that an output is wrong, but not which input elements it was computed from. Index-encoded inputs make every output element carry that information. The idea is long established in memory testing, where the data written to a memory are chosen so that a given class of faults becomes observable from outside (van de Goor, 1993). One operand is *encoded*: each of its elements holds the code

$$c(f) = (f \bmod P) + 1 \quad (4)$$

of a field f , which is either the batch index i (the same code in the whole batch) or the position $rC + c$ of element (r, c) within its $R \times C$ matrix (the same codes in every batch). The other operand is a *selection* matrix with a single one in each row or column, identical in every batch. When b is encoded, $a_{i,m,k} = [k = m \bmod K]$, and when a is encoded, $b_{i,k,n} = [k = n \bmod K]$, so that a correct product is

$$y_{i,m,n} = b_{i,m \bmod K,n} \quad \text{or} \quad y_{i,m,n} = a_{i,m,n \bmod K}, \quad (5)$$

respectively. Each output element is then a sum with one non-zero term and equals one code exactly, without rounding. The period P is the largest prime below 2^{20} in fp32 ($P = 1048573$) and below 2^{11} in fp16 ($P = 2039$), so every code is an integer that the dtype represents exactly. From a wrong output element y we decode the field that was actually read, $f' = y - 1$, and record the shift

$$\delta = f' - f \pmod{P}, \quad -P/2 \leq \delta < P/2, \quad (6)$$

against the field f that a correct product would show at that place. In fp32 the decoding is unique, because both the batch sizes and the 2^{16} positions within a matrix are below P ; in fp16 it is known only modulo 2039. With the position encoded, f' gives the row and column that were read in place of the expected ones. The two operands and the two fields give four kinds of input, which we run for the base shapes at $B_0 \pm 1$ around every candidate and at $B = 4096$ with sampled comparisons, and at the switch points with full comparisons.

Isolation. Each run executes in a fresh process, because a failure may abort the process, and a wrong result may corrupt state for later calls in the same process. A crash is recorded as an outcome. The driver appends one JSON line per run to a local file, with the configuration, the environment, the outcome, the error statistics and the timings. A manifest per sweep records the operating system build, the hardware, the Python and PyTorch versions, the tolerances and the SHA-256 hashes of the driver and the worker script. These files are released with the code.

4 Results

4.1 Boundary sweep of bmm

The main sweep on PyTorch 2.14.0 comprises 1596 runs over 42 batch sizes between 4096 and 65538, 24 of them calibration runs: 1285 are correct, 214 raise an error, and 97 are silently wrong. Here and below, the number of runs of a series includes its calibration runs, which are all correct; the comparisons with the rules and between machines exclude them, as stated there. No run crashed or timed out, and no wrong output contained a non-finite value. Sampled and full comparisons agree wherever both were run, and so do the three seeds. The largest calibration errors are $\hat{e}_{\text{fp32}} = 1.5 \times 10^{-6}$ and $\hat{e}_{\text{fp16}} = 4.2 \times 10^{-4}$, so the tolerances of Eq. (2) are $\tau_{\text{fp32}} = 1.5 \times 10^{-5}$ and $\tau_{\text{fp16}} = 4.2 \times 10^{-3}$.

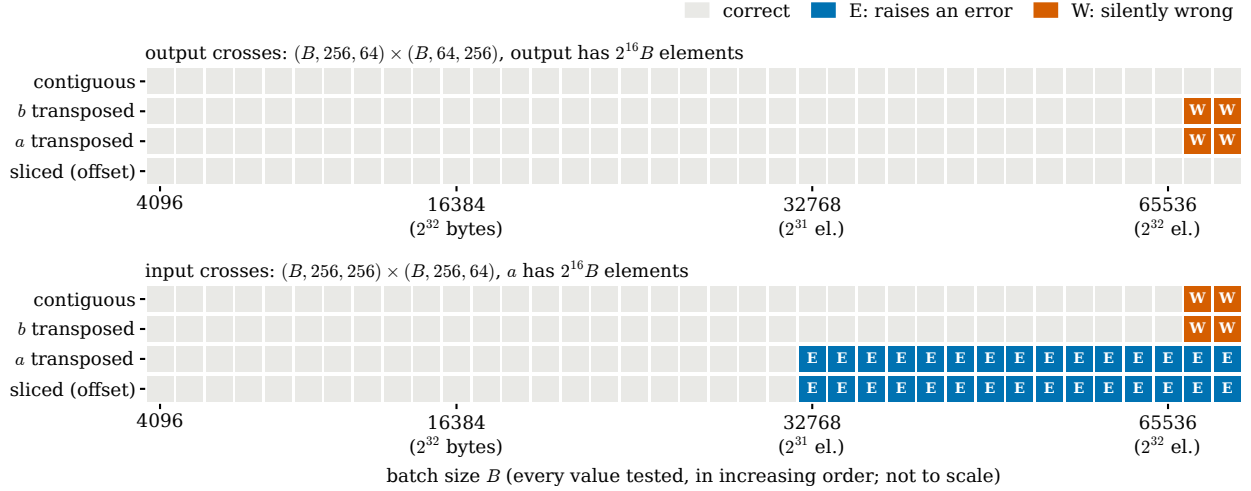


Figure 1: Boundary map of `bmm` on MPS (PyTorch 2.14.0, macOS 27.0, fp32, random inputs). Each cell is one batch size; five consecutive values are tested around each candidate boundary, and the remaining values form a grid. The fp16 results have the same pattern at the same batch sizes.

Boundary map. Figure 1 shows the two base shapes. Nothing changes at 2^{32} bytes. The silent failures start between $B = 65536$ and $B = 65537$, that is, the result is still correct at exactly 2^{32} elements and wrong one batch later. fp16 switches at the same batch sizes as fp32, so the relevant quantity is the number of elements and not the number of bytes. The exceptions start at $B = 32768$, where the view operand has exactly 2^{31} elements, and carry the message MPSGraph does not support tensor dims larger than `INT_MAX`. Every change of behavior lies between two consecutive batch sizes next to a candidate, so no bisection was triggered and the switch points are exact.

The outcome of every run with random inputs (1076 runs outside calibration, six shapes, two dtypes, four layouts) is reproduced by the rules in Table 3, applied in order. The order matters. With a of shape $(B, 512, 64)$ passed as a transposed view, the operation is correct at $B = 65535$, raises at $B = 65536$, where a reaches 2^{31} elements while the output has exactly 2^{32} , and is silently wrong at $B = 65537$, where the output exceeds 2^{32} . With sliced operands of the same shape it raises at $B = 65536$ and is correct again at $B = 65537$. A larger problem can therefore turn an explicit error into a silent failure. The rules describe what we observed on this PyTorch version and are not a specification. No configuration of the main sweep has an output above 2^{32} elements together with an input above 2^{32} , so it cannot tell whether the first rule also takes precedence over the third; Section 4.2 measures this case.

Table 3: Rules that reproduce all 1076 outcomes of the sweep with random inputs (PyTorch 2.14.0), applied in order. A *view* is an operand passed as a transposed or a sliced tensor.

	condition	outcome	extent
1	output $> 2^{32}$ elements, an operand is transposed	silently wrong	every batch of the output
	output $> 2^{32}$ elements, otherwise	correct	
2	an operand that is a view has $\geq 2^{31}$ elements	raises an error	only batches beyond element 2^{32}
3	a contiguous operand has $> 2^{32}$ elements	silently wrong	
	otherwise	correct	

Two kinds of wrong result. The two silent failures differ in extent. When the output crosses the boundary and an operand is transposed (rule 1), all 65537 batches are wrong, including batch 0, and the largest error e_i of Eq. (1) is between 1.7 and 2.2. Exceeding the boundary by a single batch thus invalidates the whole output. When a contiguous input crosses the boundary (rule 3), only the batches that lie beyond element 2^{32} of that input are wrong, with e_i between 1.4 and 1.7, and all earlier batches pass the comparison. This is the pattern of the case study in Section 4.5, where the texts before position 1365 are unaffected.

What the wrong values are. For each wrong run we tested the four hypotheses of Section 3 about how the inputs were read, and counted the wrong elements that each reproduces. Under rule 1, every wrong element equals the value obtained when the strides of the transposed operand are ignored, that is, when its storage is read as if it were

contiguous. Under rule 3, every wrong element equals the value obtained when the flat index into the large operand wraps around at 2^{32} elements, so that batch 65536 is computed from batch 0. The index-encoded inputs confirm both readings element by element without reference to a hypothesis. With the batch index encoded in a , the shift of Eq. (6) is $\delta = -65536$ for every wrong element: the decoded batch lies exactly 2^{16} batches, that is 2^{32} elements, before the expected one. In fp16 the shift is $\delta = -288$, which is -65536 modulo $P = 2039$. With the position inside the matrix encoded in a transposed b of logical shape 64×256 , the output that should contain element $(0, 1)$ contains element $(1, 0)$, and the one that should contain $(16, 0)$ contains $(0, 64)$, as expected when a 256×64 storage is read row by row as 64×256 . The three rules predict the outcome of 460 of the 496 runs with index-encoded inputs. The other 36 are predicted to be wrong but are correct, and they are explained by the inputs themselves. Some index-encoded inputs do not change under the misreading, for example values that are constant within a batch when strides are ignored, or operands that are identical in every batch when the index wraps by whole batches. We checked this on the CPU for all 64 index-encoded runs in which the rules predict a wrong result, by computing the product under the predicted misreading. It equals the correct product exactly in the 36 runs that were recorded as correct, and differs from it in the 28 runs that were recorded as wrong. With this taken into account, the rules account for all 1572 runs outside calibration.

Earlier PyTorch versions. Table 4 summarizes the reduced sweep of the base shapes on ten earlier releases ($\tau_{\text{fp32}} = 1.2 \times 10^{-5}$ and $\tau_{\text{fp16}} = 4.2 \times 10^{-3}$ in every release). The releases fall into four groups with identical outcomes within each group. Two results hold in every release since 2.5.1: a transposed operand with an output above 2^{32} elements gives a wrong output that matches the ignored-strides computation, and a contiguous input above 2^{32} elements gives wrong batches that match the wrapped index. What changed over time is how an oversized *view* is handled. In 2.4.1 it is accepted below 2^{32} elements and returns an output of zeros from 2^{32} elements on (the truncated outcome; 16 runs, all on 2.4.1). From 2.5.1 to 2.8.0 it aborts the process with a Metal assertion (`NDArray dimension length > INT_MAX`) between 2^{31} and 2^{32} elements, raises an exception at exactly 2^{32} elements, and above 2^{32} elements is silently wrong with the wrapped index. From 2.9.1 on it raises an exception everywhere from 2^{31} elements upward. For a sliced operand the thresholds up to 2.9.1 apply to the storage, which holds one batch more than the view, so they are reached one batch earlier. Version 2.4.1 differs in two further ways. With an fp32 output above 2^{32} elements it is wrong for every layout, including contiguous operands, but only in the first and the last batch, and half of the wrong elements are zero; none of our hypotheses reproduces these values. With fp16 the same configurations are correct. Across all versions no wrong output contains a non-finite value, no run is classified as input corrupt or timeout, no bisection was triggered, and within 2.10.0 to 2.14.0 we found no difference for `bmm`.

Table 4: Outcome of `bmm` by PyTorch version (base shapes, fp32 and fp16 unless noted, macOS 27.0). n is the number of elements of the tensor in question. **W**: silently wrong; **Z**: output of zeros, silently; **ok**: correct; **err**: raises; **crash**: process aborts.

condition	2.4.1	2.5.1–2.8.0	2.9.1	2.10.0–2.14.0
output $n > 2^{32}$, contiguous or sliced	W (fp32), ok (fp16)	ok	ok	ok
output $n > 2^{32}$, an operand transposed	W (fp32), ok (fp16)	W	W	W
contiguous input $n > 2^{32}$	W	W	W	W
view input, $2^{31} \leq n < 2^{32}$	ok	crash	err	err
view input, $n = 2^{32}$	Z	err	err	err
view input, $n > 2^{32}$	Z	W	err	err

4.2 Extension series

Inputs and output above 2^{32} together. With $(M, K, N) = (256, 256, 256)$, fp32 and fp16 behave identically (228 runs, and 36 further runs that repeat the wrong and the neighboring correct points with comparisons over all elements and three seeds). Contiguous operands are correct at every batch size up to $B = 65538$, although a and b then exceed 2^{32} elements; the largest error is 1.3×10^{-6} in fp32 and 4.8×10^{-4} in fp16. With a transposed operand the operation is correct up to $B = 32767$, raises the `INT_MAX` exception from $B = 32768$ to 65536, where the view has at least 2^{31} elements, and is silently wrong in every batch from $B = 65537$, where the output exceeds 2^{32} elements (24 comparisons over all elements, largest error between 2.05 and 2.19, all wrong elements reproduced by the ignored strides). With sliced operands it raises over the same range and is correct again from $B = 65537$. This is exactly what Table 3 predicts: once the output exceeds 2^{32} elements, the first rule decides the outcome, and neither the exception for large views nor the wrapped index of a large contiguous input occurs.

About $2 \cdot 2^{32}$ elements. At $B = 131071$, 131072 and 131073 the picture does not change (124 runs, all elements compared, fp32 and fp16 identical). With the output-side shape, contiguous and sliced operands are correct and a transposed operand makes every batch wrong (largest error 2.16 to 2.18). With the input-side shape and contiguous operands, the wrong batches are exactly those from 65536 to $B - 1$, that is, every batch beyond element 2^{32} of a , and every wrong element matches the index wrapped at 2^{32} ; at $B = 131073$ this includes batch 131072, which lies beyond $2 \cdot 2^{32}$ elements, so the index wraps a second time. A transposed b of shape $(B, 256, 64)$ gives the same wrong batches at $B = 131071$ and raises from $B = 131072$, where this view reaches 2^{31} elements; the second rule thus applies to either operand. A transposed or sliced a raises at all three batch sizes. Of the 24 runs with index-encoded inputs for which the rules predict a wrong result, 12 are wrong and 12 are correct, and the check of Section 4.1 shows that the 12 correct ones are those whose input does not change under the misreading.

Large storage offsets. All 104 runs of the offset series are correct in every element, in fp32 and fp16 and for both base shapes. This includes views that straddle element 2^{31} or 2^{32} of their storage and views that lie entirely beyond it; the largest storage offset is $2^{32} + 2^{17}$ elements in a storage of 4.56×10^9 elements. A small contiguous view is therefore computed correctly wherever it lies in a large device tensor, on PyTorch 2.14.0. We did not combine a large offset with a transposed view.

The tolerances of these series are $\tau_{\text{fp32}} = 1.15 \times 10^{-5}$ and $\tau_{\text{fp16}} = 3.9 \times 10^{-3}$ for the $(256, 256, 256)$ shape, and 1.23×10^{-5} and 4.2×10^{-3} for the far points and the offset series; every wrong result exceeds them by five orders of magnitude or more in fp32.

CUDA control. On the A100, the three extension series are correct in every run: 204 runs of the $(256, 256, 256)$ shape and 36 runs of its repeated points, 124 runs at the far points, and 104 runs of the offset series, with a largest error of 1.4×10^{-6} in fp32 and 4.8×10^{-4} in fp16.

The rules against all runs. Table 5 applies the rules of Table 3, unchanged, to every run with random inputs. They reproduce all 1376 outcomes. The case that the main sweep could not decide is settled by the 16 runs with contiguous operands of the $(256, 256, 256)$ shape above 2^{32} elements: a reading in which the third rule still applied there would predict wrong batches, and all 16 are correct.

Table 5: The rules of Table 3 applied to every run with random inputs (PyTorch 2.14.0, macOS 27.0; calibration runs excluded). For wrong runs with a comparison over all elements, agreement includes the extent: every batch, or exactly the batches beyond element 2^{32} .

series	runs	outcome reproduced
main sweep (Section 4.1)	1076	1076
$(256, 256, 256)$ shape, sweep	222	222
$(256, 256, 256)$ shape, repeated points	30	30
far points	48	48
all	1376	1376

4.3 Backward pass

The backward series comprises 604 runs on PyTorch 2.14.0 ($\tau_{\text{fp32}} = 1.28 \times 10^{-5}$, $\tau_{\text{fp16}} = 4.2 \times 10^{-3}$): 420 are correct in all three tensors, 40 are silently wrong in at least one, and 144 raise an exception. fp32 and fp16 agree at every point.

Correct forward pass, wrong gradients. With the output-side shape, everything is correct up to $B = 65536$. At $B = 65537$ and 65538 , with contiguous or sliced operands, the forward output is correct, as in Section 4.1, but both gradients are silently wrong (20 runs). The wrong batches of $\partial \ell / \partial a$ and $\partial \ell / \partial b$ are exactly those from 65536 on, that is, the batches beyond element 2^{32} of G ; the comparisons over all elements at $B = 65537$ with three seeds show a single wrong batch, the last one, in every layout and dtype. The largest error is between 1.16 and 1.57, and every wrong element matches the index wrapped at 2^{32} . With a transposed operand, the forward output is wrong in every batch, as before, and the gradients are wrong in the same last batches (20 runs). A model whose forward pass is correct at this size can therefore still be trained on wrong gradients, without an error or a non-finite value.

Exceptions. With the input-side shape, everything is correct up to $B = 32767$, and every run from $B = 32768$ raises the INT_MAX exception. For a transposed or sliced a it is raised in the forward pass, as in Section 4.1. For contiguous

operands and for a transposed b , the forward pass succeeds and the exception is raised in the backward pass (72 of the 144 runs). At these sizes the failure is loud.

The same rules. These outcomes are what Table 3 gives when it is applied to the two products of Eq. (3), with G as a contiguous operand and each transpose as a view: the transpose of a contiguous operand is a transposed view, and the transpose of an operand that was passed as a transposed view is its contiguous storage. For the output-side shape, G is a contiguous operand with more than 2^{32} elements from $B = 65537$ while the gradients themselves stay below 2^{32} elements, which is the third rule. For the input-side shape, the transpose of a is a view with 2^{31} elements at $B = 32768$, which is the second rule. Applied in this way, the rules reproduce the class of the output and of both gradients in all 448 runs that return a result, including the extent of the wrong batches where all elements were compared, and the pass in which the exception occurs in all 144 runs that raise. We state this as an agreement between outcomes and rules; how the backward pass forms its operands was not observed.

CUDA control. On the A100, all 568 runs of the backward series are correct in all three tensors (24 of them calibration runs). This includes the batch sizes $B = 65537$ and 65538 of the output-side shape, where MPS returns wrong gradients, and the runs of the input-side shape from $B = 32768$, which raise on MPS. The largest error is 1.5×10^{-6} , 1.4×10^{-6} and 1.4×10^{-6} for the output and the two gradients in fp32, and at most 4.9×10^{-4} for each of them in fp16. The series has fewer runs than on MPS (604) because there are no switch points to compare on both sides.

4.4 A second machine and two macOS versions

Machine B repeats all 6348 runs of machine A under each of its two macOS versions, 6156 of them outside calibration. Table 6 compares the three configurations run by run, where a run is identified by its operation, shape, dtype, layout, input, batch size, seed and kind of comparison, and in the offset series also by the number of leading batches. In each of the three pairs, every run of one configuration has its counterpart in the other, and in all 6156 the two agree in every compared field: the class, the largest error of Eq. (1) as a floating-point value, the ranges of wrong batches, the number of wrong elements, the fractions of wrong elements that each hypothesis reproduces, the shifts decoded from index-encoded inputs, the message of each exception, and, for the backward series, the same quantities per tensor and the pass in which an exception is raised. The calibration errors, and hence the tolerances, are identical as well. The agreement covers the wrong results as much as the correct ones. It includes the behavior that is specific to old PyTorch versions: the outputs of zeros and the wrong values of 2.4.1 that no hypothesis reproduces, and the process aborts of 2.5.1 to 2.8.0. The comparison is produced by a script that is released with the raw results and fails on any difference.

Table 6: Machine A (M2 Ultra, macOS 27.0) and machine B (M2 Max) under macOS 26.6.2 and under macOS 27.0, compared run by run, calibration runs excluded. The counts are the same in all three pairs of configurations. “Identical” means equal in every compared field, among them the class, the largest error, the wrong batches, the hypothesis fractions and the tolerance.

series	PyTorch	common runs	identical	only in one configuration
main sweep	2.14.0	1572	1572	0
reduced sweep	2.4.1, 2.9.1–2.13.0 (each)	348	348	0
reduced sweep	2.5.1–2.8.0 (each)	362	362	0
inputs and output together	2.14.0	252	252	0
far points	2.14.0	112	112	0
large storage offsets	2.14.0	92	92	0
backward pass	2.14.0	592	592	0
all		6156	6156	0

The failures are therefore not a property of one machine, of the M2 Ultra, or of one macOS version, and they are deterministic to the last bit of the reported error. The three configurations separate the two factors. Machine A and machine B under macOS 27.0 run the same build of the operating system on different chips, and the two passes of machine B run different macOS versions on the same hardware; neither changes any result. The main sweep takes 116 and 118 minutes on machine B and 114 minutes on machine A, although machine B has half the GPU cores, because the run time is dominated by generating the inputs and computing the float64 reference on the CPU.

4.5 Case study: sentiment classification with eager attention

To check that the failure reaches real workloads, we ran a public sentiment classifier through the standard Hugging Face implementation. The model is a RoBERTa-base encoder (Liu et al., 2019) with 12 attention heads that was

pretrained on tweets and fine-tuned for three-class sentiment analysis in the TimeLMs project (Loureiro et al., 2022). We run it with `attn_implementation="eager"`, in fp32 under `torch.inference_mode()`². The inputs are the first 2048 tweets of the sentiment test split of the TweetEval benchmark (Barbieri et al., 2020) (revision b3a375ba), tokenized once with padding to $L = 512$ and reused unchanged in every condition. Environment: the machine of Table 1 with macOS 27.0, PyTorch 2.14.0 and Transformers 5.17.0.

Reference. In inference mode a Transformer has no interaction between samples, so running the batch in one call must give the same outputs as running it in pieces, $f(\text{concat}(X_1, \dots, X_k)) = \text{concat}(f(X_1), \dots, f(X_k))$. We use this metamorphic relation as the reference: the same model on MPS with chunks of 256 texts, whose attention scores have 8.1×10^8 elements. We checked the reference against CPU fp32 on 1000 texts, including the first, middle and last 100, texts 1344–1384 around the predicted boundary, and the texts with the smallest margin between the top two logits. The largest relative logit difference was 6×10^{-5} , and no prediction changed.

Batch sizes. Attention scores have $B \cdot 12 \cdot 512^2$ elements, so $B = 1365$ is just below 2^{32} and $B = 1366$ is just above it. $B = 2048$ gives 1.5×2^{32} . At $L = 512$ the feed-forward activations stay below 2^{32} at all three sizes, so only the attention products cross the boundary. We call a text corrupted when its relative logit error exceeds 10^{-3} .

Table 7: Sentiment classification on MPS (PyTorch 2.14.0, macOS 27.0) with the whole batch in one forward pass, compared with chunked execution. “Scores” is the number of attention-score elements minus 2^{32} . Accuracy is on the three-class TweetEval labels; the value in parentheses is the reference.

B	attention	scores $- 2^{32}$	corrupted texts	changed predictions	max. rel. error	accuracy
1365	eager	$-1,048,576$	0	0	0	0.708 (0.708)
1366	eager	$+2,097,152$	1	1	0.78	0.707 (0.708)
1366	SDPA	$+2,097,152$	0	0	1.9×10^{-5}	0.708 (0.708)
2048	eager	$+2^{31}$	683 (33.4%)	386 (18.9%)	5.16	0.639 (0.713)
2048	SDPA	$+2^{31}$	0	0	1.9×10^{-5}	0.713 (0.713)

Results. Table 7 and Figure 2 show the outcome. Every run finished normally, with no error, warning, NaN or Inf. At $B = 1365$ the single call matches the reference bit for bit. At $B = 1366$ exactly one text, the last one, is corrupted and its prediction changes. At $B = 2048$ texts 1365–2047 are corrupted, 386 predictions (18.9%) change, and accuracy drops from 0.713 to 0.639. Every text before position 1365 still matches the reference bit for bit. Fused SDPA on the same batches agrees with the reference to within 1.9×10^{-5} .

Effect on predictions. At $B = 2048$ the two groups of positions do not overlap: all 1365 texts before position 1365 have a relative error of exactly zero, and all 683 texts from position 1365 onward exceed the threshold, with a minimum of 0.10 and a median of 0.91. The corrupted outputs are not spread over the classes. Of the 683 corrupted texts, 661 are predicted as *neutral*, whereas the reference assigns 242, 297 and 144 texts to negative, neutral and positive. Accuracy on these 683 texts falls from 0.723 to 0.501, which is what a constant *neutral* prediction would score, since 344 of them (0.504) carry that label. The reference is correct and the single call wrong on 266 texts, and the reverse holds on 114 (exact McNemar test, $p = 4.1 \times 10^{-15}$). The damage is therefore much larger per class than overall. Over all 2048 texts, accuracy drops by 7.4 points, while recall drops from 0.788 to 0.529 for negative and from 0.740 to 0.504 for positive, and rises from 0.656 to 0.758 for neutral; on the 683 corrupted texts, recall for negative and positive is 0.010 and 0.054. Studies of nondeterminism in training observe the same pattern, with per-class and subgroup metrics varying far more than overall accuracy (Pham et al., 2020; Zhuang et al., 2021), although the cause there is rounding-level variation amplified by training and here it is an order-one error in a single forward pass. Because the corrupted outputs collapse onto one class, the visible loss depends on the label distribution, and a data set dominated by that class could hide it.

Where the corruption starts. One head’s score matrix has $512^2 = 2^{18}$ elements, so 2^{32} elements correspond to $2^{14} = 16384 = 1365 \times 12 + 4$ head matrices. If positions past element 2^{32} of the flattened scores are affected, corruption should begin at head 4 of text 1365. Comparing the intermediate tensors of the first attention layer with the reference, the scores $QK^\top$ and the softmax output agree bit for bit at every batch size, and the first corrupted (text, head) pair in the product $\text{softmax}(QK^\top)V$ is (1365, 4), with heads 4–11 of text 1365 corrupted. Feeding the reference softmax output into that product alone reproduces the same corruption. This matches the layer-level result that `bmm` fails when an input exceeds 2^{32} elements.

²<https://huggingface.co/cardiffnlp/twitter-roberta-base-sentiment-latest> (revision 3216a57f)

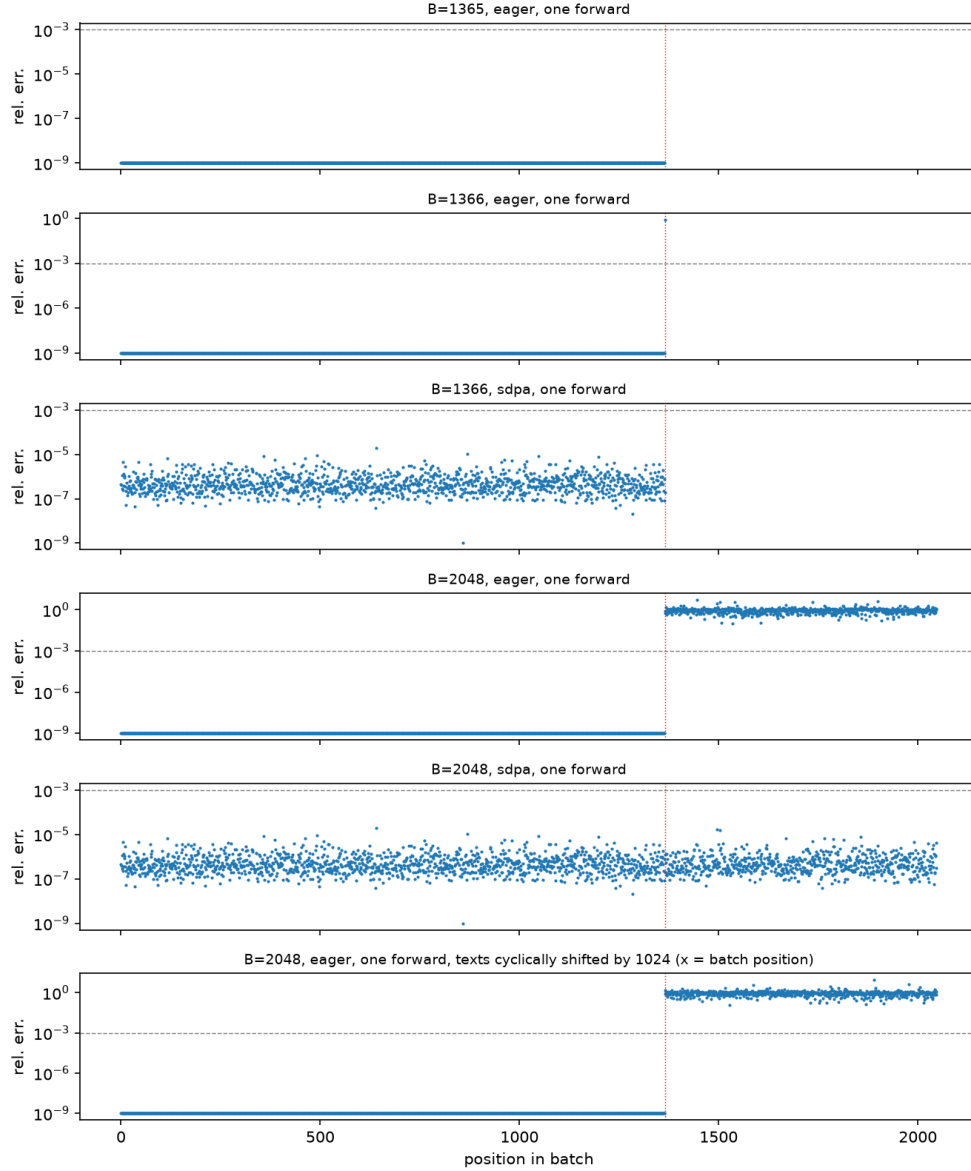


Figure 2: Relative logit error of each text against the chunked reference, by position in the batch. Exact agreement is drawn at 10^{-9} . The dotted red line marks text 1365, the first text predicted to cross 2^{32} score elements; the dashed gray line is the 10^{-3} threshold. The last panel runs the same 2048 texts cyclically shifted by 1024.

Position, not content. We repeated $B = 2048$ with the texts cyclically shifted by 1024. The corrupted batch positions are again 1365–2047, and the corrupted texts (original indices 341–1023) do not overlap with those of the unshifted run. Whether a text is misclassified depends on where it sits in the batch, not on what it says.

A loud failure at $L = 256$. We first tried $L = 256$. There the feed-forward activation $(B \cdot 256) \times 3072$ crosses 2^{32} at the same $B = 5462$ as the scores, and the process aborted inside a Metal matrix-multiplication assertion (destination [3072, 1398272] is too large for kernel), while $B = 5461$ matched the reference exactly. Whether a given oversized operation fails loudly or silently therefore depends on which operation crosses the boundary first.

4.6 Other operations and CUDA

First-pass suite. Before the sweep we ran a suite of 26 test cases on all 11 PyTorch releases, with one fixed shape per case and a comparison against the CPU at sampled batch indices. Table 8 shows the cases that involve matrix multiplication and attention. The `bmm` rows agree with the sweep, and the suite adds two observations. Eager attention is silently wrong from 2.5.1 onward, as expected from the `bmm` results, except on 2.13.0, where the process aborts. On 2.4.1 it is correct at this shape, which the `bmm` rows alone do not explain. Fused SDPA is silently wrong up to 2.12.1 when its scores exceed 2^{32} elements and correct from 2.13.0. A dispatch trace shows that on MPS fused SDPA resolves to `aten::scaled_dot_product_attention_math_for_mps` in both 2.12.1 and 2.14.0, and not to the FlashAttention operators that PyTorch provides for CUDA.

Table 8: First-pass results on macOS 27.0 by PyTorch version. **W**: silently wrong (relative error against CPU above 10^{-2}); **ok**: correct; **err**: raises; **crash**: process aborts. Output-side cases use $(B, 256, 64) \times (B, 64, 256)$ with $B = 65552$ or 68544; the input-side case uses $(B, 256, 256) \times (B, 256, 64)$ with $B = 65552$; attention uses scores of shape (5712, 12, 256, 256).

case	2.4.1	2.5.1	2.6.0	2.7.1	2.8.0	2.9.1	2.10.0	2.11.0	2.12.1	2.13.0	2.14.0
<code>bmm</code> , output $> 2^{32}$, contiguous	W	ok	ok	ok	ok	ok	ok	ok	ok	ok	ok
<code>bmm</code> , output $> 2^{32}$, b transposed	W	W	W	W	W	W	W	W	W	W	W
<code>bmm</code> , output $> 2^{32}$, a transposed	W	W	W	W	W	W	W	W	W	W	W
<code>bmm</code> , input $> 2^{32}$, contiguous	W	W	W	W	W	W	W	W	W	W	W
eager attention	ok	W	W	W	W	W	W	W	W	crash	W
fused SDPA	W	W	W	W	W	W	W	W	W	ok	ok

Regression in 2.14.0. On 2.14.0, `torch.arange` with $N = 2^{32} + 2^{20}$ elements writes only the first $N \bmod 2^{32}$ elements and leaves the rest zero without an error. Versions 2.12.1 and 2.13.0 raise instead (MPSGraph does not support tensor dims larger than `INT_MAX`). This is the only regression that we found in 2.14.0; for `bmm` the sweep shows no difference between 2.10.0 and 2.14.0 (Table 4).

CUDA as a control. Correctness in this paper is always judged against float64 on the CPU. CUDA serves a different purpose: it shows whether a failure belongs to the operation at that size or to the MPS backend. The controls for the backward and the extension series are reported with those series (Sections 4.2 and 4.3). The main sweep on the A100 (PyTorch 2.14.0 with CUDA 13.0) comprises 1494 runs in 18 chunks, 54 of them calibration runs, over the same six shapes, two dtypes, four layouts and 42 batch sizes, with 992 runs on random and 448 on index-encoded inputs outside calibration. All 1494 runs are correct: no run is wrong, raises, crashes or returns zeros. This includes the 288 runs in which a tensor has more than 2^{32} elements, 160 of them compared over all elements, which are the configurations where MPS fails. The largest error over all runs is 1.8×10^{-6} in fp32 and 4.8×10^{-4} in fp16, well below the tolerance of every chunk (6.0×10^{-6} to 1.5×10^{-5} in fp32, 3.8×10^{-3} and 4.2×10^{-3} in fp16) and of the MPS sweep. Of the 1314 runs that the two sweeps have in common, 203 have a different outcome, and in all of them MPS raises (159) or is silently wrong (44) where CUDA is correct. The number of runs is lower than on MPS (1596) only because there are no switch points to compare on both sides. Before the sweep, we had also run the first-pass `bmm` configurations on the same GPU model with PyTorch 2.11.0 and CUDA 12.8 (48 configurations, all correct, largest error 9.2×10^{-7} in fp32 and 4.5×10^{-4} in fp16 against float64 on the GPU, with spot checks against the CPU), together with eager attention and fused SDPA with scores of shape (5712, 12, 256, 256), which are correct with errors below 10^{-6} . The `bmm` failures are therefore specific to the MPS backend and are not inherent to the operation, the shapes or the memory layouts. CUDA is not free of problems at this boundary, however. On the same A100, `torch.arange` with $N = 2^{32} + 2^{20}$ int64 elements leaves every element from index 2^{32} onward at zero, without an error, in all four versions we tried (2.11.0 to 2.14.0) (Niimi, 2026a). A fix was proposed within a day of the report; its author attributes the failure to an index that is multiplied in 32 bits before it is widened, in a kernel that `arange` shares with `range`, `linspace` and `logspace`.³ MPS on 2.14.0 fails differently on the same call, writing only the first $N \bmod 2^{32}$ elements. Two consequences follow. A second GPU backend cannot serve as the oracle for a differential test at this scale, which is why we use the CPU in float64 and, for `arange`, the closed form $x_i = i$. And silent failures at 2^{32} elements are not confined to one backend, which suggests that this size range is rarely exercised by existing tests.

³<https://github.com/pytorch/pytorch/pull/197713>, open at the time of writing. We did not examine the source ourselves.

5 Mitigation

One rule. A result that is silently wrong is worse than an error, so the mitigation should turn the first into the second. The rules of Table 3 could in principle be used to let the safe cases through, but Table 4 shows that they change between versions, and Table 8 shows that attention and fused SDPA fail on their own schedule. We therefore use a single rule that refers to neither the version nor the operation: *do not create or read an MPS tensor with 2^{32} or more elements*. Large batches are split into chunks before they reach the device, as in the chunked reference of Section 4.5. `bmm`, and attention with it, has no interaction between batches, so chunking does not change the result. The same rule covers training. In the backward series the gradients are wrong only where the upstream gradient, which has the size of the forward output, exceeds 2^{32} elements (Section 4.3); a forward pass that respects the rule never produces such a gradient, and gradients accumulated over chunks equal those of the whole batch when the loss is a sum over samples. The offset series (Section 4.2) suggests that contiguous chunks could also be taken from a large tensor that is already on the device, but this was measured on one version only, and the rule does not rely on it.

Guard. The released guard enforces the rule. It is a `TorchDispatchMode` that inspects the inputs and outputs of every aten operation on MPS and raises an exception when one of them has 2^{32} or more elements. For `mm`, `bmm`, `addmm`, `baddbmm` and fused SDPA it estimates the size before the operation runs, from the operand shapes (for SDPA, from the shape of the scores), because some versions abort the process inside the operation. For `bmm` the message states how many batches per chunk keep every operand and the output below the limit. The guard is installed with one call and needs no change to the model code.

Evaluation against the sweep. We applied the stopping rule to the recorded outcome of every run (Table 9). Over all 11 versions, 481 of 5252 runs are silently wrong or return zeros, and the rule stops all of them. The threshold has to be inclusive: with “more than 2^{32} ” the rule would pass 16 runs on 2.4.1 in which a view of exactly 2^{32} elements returns zeros. In no version is a run below 2^{32} elements silently wrong. The price is that the rule also stops 630 of the 4095 correct runs, namely those in which a tensor of 2^{32} or more elements happens to be handled correctly by that version. The rule fares the same on the series of Sections 4.2 and 4.3, where the size of a run includes the upstream gradient, the gradients and the storage behind an offset view: all 104 silent runs are stopped, among them the 20 runs of the backward series in which only the gradients are wrong, and 128 of 730 correct runs are stopped as well, 28 of them offset runs whose storage reaches 2^{32} elements. Run on the first-pass suite, the guard itself leaves no silent failure and no crash on 2.11.0, 2.12.1 and 2.14.0, and on the backward case with contiguous operands at $B = 65537$ it raises before the forward product is computed.

Table 9: The stopping rule applied to the recorded outcomes of every series on machine A. “Silent” counts runs that are silently wrong or return zeros; for the backward series, in any of the three tensors.

PyTorch	runs	silent runs	silent runs stopped		correct runs stopped
			$n \geq 2^{32}$	$n > 2^{32}$	
2.4.1	360	56	56	40	40 of 304
2.5.1–2.8.0 (each)	374	42	42	42	48 of 282
2.9.1	360	32	32	32	40 of 274
2.10.0–2.13.0 (each)	360	32	32	32	40 of 276
2.14.0 (main sweep)	1596	97	97	97	198 of 1285
all versions, main and reduced sweeps	5252	481	481	465	630 of 4095
2.14.0, (256, 256, 256) shape	264	32	32	32	28 of 154
2.14.0, far points	124	32	32	32	40 of 52
2.14.0, offset series	104	0	0	0	28 of 104
2.14.0, backward pass	604	40	40	40	32 of 420

Overhead. The guard adds Python work to every operation. On a small training loop (a two-layer network on 1024×120 inputs, 2000 steps) it increases the run time by a factor of 1.73, and on large attention forward passes (batch 64, 32 heads, length 1024, fp16) by a factor of 1.00. The cost is thus negligible in the workload with large tensors, where the guard is needed, and the guard can be left out in workloads whose tensors are small by construction.

6 Conclusion

6.1 Key Findings

- **Silent failures beyond 2^{32} elements follow three rules.** `bmm` on the MPS backend returns wrong values without an error in all 11 PyTorch releases from 2.4.1 to 2.14.0. Three rules, applied in order, reproduce all 1376 outcomes with random inputs. When the output exceeds 2^{32} elements, it alone decides the outcome: a transposed operand makes every batch wrong, equal to a computation that ignores its strides, and anything else is correct. Otherwise a view with 2^{31} or more elements raises an exception, and a contiguous input above 2^{32} elements makes exactly the batches beyond that point wrong, equal to a computation whose index wraps around at 2^{32} . A slightly larger problem can therefore turn an explicit error into a silent failure. The limit is on the number of elements and not on bytes, it persists near $2 \cdot 2^{32}$ elements, and index-encoded inputs confirm both misreadings element by element.
- **Training is affected where inference is not.** With contiguous operands and an output just above 2^{32} elements, the forward pass is correct and both gradients are silently wrong in the batches beyond element 2^{32} . No new rule is needed: the rules, applied to the two products that define the gradients, reproduce all 592 outcomes of the backward sweep, including the pass in which each exception is raised.
- **The failures are deterministic and specific to the backend.** An M2 Max under macOS 26.6.2, the same machine under macOS 27.0 and an M2 Ultra under macOS 27.0 agree in all 6156 common runs of every series, down to the largest error, for correct and wrong results alike; neither the chip nor the step from macOS 26 to 27 changes any result. The same sweeps, including the backward pass, are correct on CUDA in all 2530 runs. CUDA is nevertheless not a safe oracle at this scale, because `torch.arange` is silently wrong above 2^{32} elements there as well, in a different way.
- **Real workloads are affected, and aggregate metrics hide it.** With eager attention, one oversized batch of a public sentiment classifier corrupts a third of the outputs. The corrupted predictions collapse onto one class, so overall accuracy falls by 7 points while recall for the other two classes falls to almost zero on the affected inputs. Which texts are affected depends on their position in the batch and not on their content.
- **One rule is enough in practice.** Do not create or read an MPS tensor with 2^{32} or more elements, and split large batches before they reach the device. Applied to the recorded outcomes of all 11 versions, this rule stops all 481 silent runs (wrong values or zeros), and no run below 2^{32} elements is silently wrong. It also prevents the backward failures, because the forward output, which has the size of the upstream gradient, is stopped first. The threshold must be inclusive, because 2.4.1 returns zeros for a view of exactly 2^{32} elements.

6.2 Limitations

The measurements come from two machines of one chip generation and from one operation, and several limits follow from that.

Environment. The MPS results come from two machines of the M2 generation, with macOS 26.6.2 and 27.0. We did not run macOS 14 or 15. That the behavior begins with macOS 15 rests on the error message that PyTorch raised before that release, and not on our own measurements. Other chip generations are untested. Repeating the sweep on older macOS versions is left for future work.

Coverage. The systematic sweep covers `bmm` only. Other operations are covered by the first-pass suite with one shape each, so for them we know that a failure exists, but not where it begins. The sweep uses `fp32` and `fp16` and does not include `bfloat16`. Beyond 2^{32} elements it has points just above the boundary and near $2 \cdot 2^{32}$, and nothing in between except the case study at 1.5 times 2^{32} . The backward series, the (256, 256, 256) shape, the far points and the offset series use PyTorch 2.14.0 only. The backward series tests the gradients of a single `bmm`, not a training run. The offset series does not combine a large offset with a transposed view. The sweep of earlier PyTorch versions uses a coarser grid, one seed for the full comparisons and the base shapes only.

Interpretation. The rules in Table 3 summarize observed outcomes for one PyTorch version. They agree with every run, including the extension and backward series, which cover conditions outside the sweep from which the rules were derived, but they describe a finite set of configurations and may not hold outside it. That the rules also reproduce the backward series rests on an assumption about how the two gradient products receive their operands, which we did not observe. The CUDA control covers every series on one GPU model and one PyTorch version (2.14.0); earlier versions were run on CUDA only for the first-pass configurations (2.11.0).

Case study and guard. The case study uses one model, one data set and one input length. The collapse of the corrupted predictions onto a single class is an observation on this example, and other models may fail differently. The guard was run on the first-pass suite, and its stopping rule was checked against the recorded outcomes of the sweep; these are the same operations that motivated it. Its overhead was measured on two workloads. We recommend building the chunks before they reach the device, as in the case study. Slicing contiguous chunks out of a device tensor that already has 2^{32} or more elements was correct in the offset series, but only on one PyTorch version, and creating such a tensor is what the rule forbids.

Acknowledgments

This study is supported by JSPS KAKENHI (Grant No. JP24K16472).

References

- Abnar, S. and Zuidema, W. (2020). Quantifying attention flow in Transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4190–4197. Association for Computational Linguistics.
- Bahdanau, D., Cho, K., and Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. In *3rd International Conference on Learning Representations (ICLR)*. arXiv:1409.0473.
- Barbieri, F., Camacho-Collados, J., Espinosa Anke, L., and Neves, L. (2020). TweetEval: Unified benchmark and comparative evaluation for tweet classification. In *Findings of the Association for Computational Linguistics: EMNLP 2020*.
- Chen, J., Liang, Y., Shen, Q., Jiang, J., and Li, S. (2023). Toward understanding deep learning framework bugs. *ACM Transactions on Software Engineering and Methodology*.
- Chen, T. Y., Kuo, F.-C., Liu, H., Poon, P.-L., Towey, D., Tse, T. H., and Zhou, Z. Q. (2018). Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 51(1).
- Clark, K., Khandelwal, U., Levy, O., and Manning, C. D. (2019). What does BERT look at? An analysis of BERT’s attention. In *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 276–286. Association for Computational Linguistics.
- Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems 35*, pages 16344–16359.
- Deng, Y., Xia, C. S., Peng, H., Yang, C., and Zhang, L. (2023). Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*.
- Dietz, W., Li, P., Regehr, J., and Adve, V. (2015). Understanding integer overflow in C/C++. *ACM Transactions on Software Engineering and Methodology*, 25(1):1–29.
- Dixit, H. D., Pendharkar, S., Beadon, M., Mason, C., Chakravarthy, T., Muthiah, B., and Sankar, S. (2021). Silent data corruptions at scale. *arXiv preprint arXiv:2102.11245*.
- Hammond, J. R., Schäfer, A., and Latham, R. (2014). To INT_MAX... and beyond! Exploring large-count support in MPI. In *2014 Workshop on Exascale MPI at Supercomputing Conference*, pages 1–8. IEEE.
- Hewitt, J. and Manning, C. D. (2019). A structural probe for finding syntax in word representations. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4129–4138. Association for Computational Linguistics.
- Hochschild, P. H., Turner, P., Mogul, J. C., Govindaraju, R., Ranganathan, P., Culler, D. E., and Vahdat, A. (2021). Cores that don’t count. In *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS)*.
- Liu, J., Huang, Y., Wang, Z., Ma, L., Fang, C., Gu, M., Zhang, X., and Chen, Z. (2023a). Generation-based differential fuzzing for deep learning libraries. *ACM Transactions on Software Engineering and Methodology*.
- Liu, J., Lin, J., Ruffy, F., Tan, C., Li, J., Panda, A., and Zhang, L. (2023b). NNSmith: Generating diverse and valid test cases for deep learning compilers. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 2*.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. (2019). RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*.

- Loureiro, D., Barbieri, F., Neves, L., Espinosa Anke, L., and Camacho-Collados, J. (2022). TimeLMs: Diachronic language models from Twitter. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*.
- Niimi, J. (2026a). [CUDA] ‘torch.arange’ silently leaves every element from index 2^{32} onward as zero when $N > 2^{32}$. PyTorch GitHub issue #197673, <https://github.com/pytorch/pytorch/issues/197673>. Opened 2026-09-19.
- Niimi, J. (2026b). [MPS] ‘bmm’ silently returns wrong results when the output exceeds 2^{32} elements and an input is non-contiguous. PyTorch GitHub issue #197636, <https://github.com/pytorch/pytorch/issues/197636>. Opened 2026-09-19.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E. Z., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *arXiv preprint arXiv:1912.01703*.
- Pham, H. V., Lutellier, T., Qi, W., and Tan, L. (2019). CRADLE: Cross-backend validation to detect and localize bugs in deep learning libraries. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*.
- Pham, H. V., Qian, S., Wang, J., Lutellier, T., Rosenthal, J., Tan, L., Yu, Y., and Nagappan, N. (2020). Problems and opportunities in training deep learning software systems: An analysis of variance. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.
- Tambon, F., Nikanjam, A., An, L., Khomh, F., and Antoniol, G. (2023). Silent bugs in deep learning frameworks: an empirical study of Keras and TensorFlow. *Empirical Software Engineering*.
- Tay, Y., Dehghani, M., Bahri, D., and Metzler, D. (2022). Efficient Transformers: A survey. *ACM Computing Surveys*, 55(6):1–28.
- van de Goor, A. J. (1993). Using march tests to test SRAMs. *IEEE Design & Test of Computers*, 10(1):8–14.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30:5998–6008.
- Wang, G., Wang, Z., Chen, J., Chen, X., and Yan, M. (2022). An empirical study on numerical bugs in deep learning programs. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.
- Wei, A., Deng, Y., Yang, C., and Zhang, L. (2022). Free lunch for testing: Fuzzing deep-learning libraries from open source. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., and Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45.
- Wressnegger, C., Yamaguchi, F., Maier, A., and Rieck, K. (2016). Twice the bits, twice the trouble: Vulnerabilities induced by migrating to 64-bit platforms. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 541–552.
- Yang, C., Deng, Y., Yao, J., Tu, Y., Li, H., and Zhang, L. (2023). Fuzzing automatic differentiation in deep-learning libraries. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 1174–1186.
- Zhuang, D., Zhang, X., Song, S. L., and Hooker, S. (2021). Randomness in neural network training: Characterizing the impact of tooling. *arXiv preprint arXiv:2106.11872*.